

Python Commands & Programs Reference

Syntax, Built-ins, and Worked Programs for Teaching

Compiled July 2026

This guide covers Python syntax fundamentals, data types, control flow, functions, object-oriented programming, file handling, exceptions, and commonly used built-in functions and modules — followed by a set of complete worked programs. Each command entry includes a plain-language description and a runnable example with its output, making it suitable for classroom hand-outs, lab exercises, or self-study.

Contents

1. Getting Started & Variables
2. Data Types & Type Conversion
3. Operators
4. Strings & String Methods
5. Lists, Tuples, Sets & Dictionaries
6. Control Flow (if / loops)
7. Functions & Lambda
8. Comprehensions
9. Exception Handling
10. File Handling
11. Object-Oriented Programming
12. Modules & Common Built-ins
13. Worked Example Programs

1. Getting Started & Variables

Python is an interpreted, dynamically typed language known for readable syntax. Statements do not require semicolons, and indentation (not braces) defines code blocks.

print()

Outputs text or values to the console. Multiple arguments are separated by the sep parameter (a space by default) and end with the end parameter (a newline by default).

EXAMPLE

```
print("Hello, World!")
print("a", "b", "c", sep="-")
```

OUTPUT

```
Hello, World!
a-b-c
```

Variables & Assignment

Variables are created the moment a value is assigned; no type declaration is needed. Python infers the type automatically.

EXAMPLE

```
name = "Ana"
age = 25
height = 1.65
is_student = True
```

input()

Pauses the program and reads a line of text typed by the user, always returning it as a string.

EXAMPLE

```
name = input("Enter your name: ")
print("Hello,", name)
```

OUTPUT

```
Enter your name: Ana
Hello, Ana
```

Comments

Text following # is ignored by the interpreter. Triple-quoted strings are often used for multi-line comments or docstrings.

EXAMPLE

```
# This is a single-line comment
"""
This is a
multi-line comment.
"""
```

2. Data Types & Type Conversion

Core Data Types

Python's built-in types cover numbers, text, and boolean logic.

COMMON TYPES

Type	Description
int	Whole numbers, e.g. 10, -3
float	Decimal numbers, e.g. 3.14
str	Text, written in quotes
bool	True or False
NoneType	Represents 'no value', written None

EXAMPLE

```
age = 25          # int
price = 9.99      # float
name = "Ana"      # str
is_valid = True   # bool
```

type()

Returns the data type of a given value or variable, useful for debugging.

EXAMPLE

```
print(type(25))
print(type("hello"))
print(type(3.14))
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Type Conversion

Functions like `int()`, `float()`, and `str()` convert a value from one type to another.

EXAMPLE

```
x = "10"
y = int(x) + 5
print(y)
print(str(3.14) + " meters")
```

OUTPUT

```
15
3.14 meters
```

3. Operators

Arithmetic Operators

Perform mathematical calculations. // is floor (integer) division and % returns the remainder.

EXAMPLE

```
print(7 + 3)    # 10
print(7 - 3)    # 4
print(7 * 3)    # 21
print(7 / 3)    # 2.333...
print(7 // 3)   # 2
print(7 % 3)    # 1
print(7 ** 2)   # 49
```

Comparison Operators

Compare two values and return a boolean result: == , != , > , < , >= , <=

EXAMPLE

```
print(5 == 5)   # True
print(5 != 3)   # True
print(5 > 3)    # True
```

OUTPUT

```
True
True
True
```

Logical Operators

Combine boolean expressions: and requires both sides true, or requires at least one, not inverts the result.

EXAMPLE

```
age = 20
has_id = True
print(age >= 18 and has_id)
print(age < 18 or has_id)
print(not has_id)
```

OUTPUT

```
True
True
False
```

Assignment Operators

Shorthand operators combine an arithmetic operation with assignment.

EXAMPLE

```
x = 10
x += 5    # x = x + 5 -> 15
x -= 3    # 12
x *= 2    # 24
x //= 5   # 4
```

4. Strings & String Methods

String Basics & Slicing

Strings are sequences of characters that support indexing (starting at 0) and slicing with [start:stop:step].

EXAMPLE

```
s = "Hello, Python"
print(s[0])      # H
print(s[7:])     # Python
print(s[::-1])   # nohtyP ,olleH
```

OUTPUT

```
H
Python
nohtyP ,olleH
```

f-strings

The preferred way to embed variables and expressions directly inside a string, prefixed with f before the quotes.

EXAMPLE

```
name = "Ana"
age = 25
print(f"{name} is {age} years old.")
print(f"Next year: {age + 1}")
```

OUTPUT

```
Ana is 25 years old.
Next year: 26
```

Common String Methods

Strings offer many built-in methods for cleaning and transforming text.

EXAMPLE

```
s = " Hello World "
print(s.strip())
print(s.upper())
print(s.lower())
print(s.replace("World", "Python"))
print("a,b,c".split(","))
print("-".join(["a", "b", "c"]))
```

OUTPUT

```
Hello World
HELLO WORLD
hello world
Hello Python
['a', 'b', 'c']
a-b-c
```

len()

Returns the number of characters in a string (or items in any sequence).

EXAMPLE

```
print(len("Python"))
```

OUTPUT

6

5. Lists, Tuples, Sets & Dictionaries

List

An ordered, mutable (changeable) collection that can hold mixed data types and allows duplicates.

EXAMPLE

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
fruits.remove("banana")
fruits[0] = "apricot"
print(fruits)
```

OUTPUT

```
['apricot', 'cherry', 'date']
```

Tuple

An ordered, immutable collection — once created, its items cannot be changed. Useful for fixed groups of values like coordinates.

EXAMPLE

```
point = (10, 20)
x, y = point
print(x, y)
```

OUTPUT

```
10 20
```

Set

An unordered collection of unique values; duplicates are automatically removed.

EXAMPLE

```
nums = {1, 2, 2, 3, 3, 3}
nums.add(4)
print(nums)
```

OUTPUT

```
{1, 2, 3, 4}
```

Dictionary

A collection of key-value pairs, allowing fast lookup of a value by its unique key.

EXAMPLE

```
student = {"name": "Ana", "age": 25}
student["grade"] = "A"
print(student["name"])
print(student.keys())
print(student.values())
```

OUTPUT

```
Ana
dict_keys(['name', 'age', 'grade'])
dict_values(['Ana', 25, 'A'])
```

Common List Methods

Frequently used operations for adding, sorting, and inspecting a list.

EXAMPLE

```
nums = [4, 1, 3]
nums.sort()
print(nums)
print(len(nums))
print(3 in nums)
```

OUTPUT

```
[1, 3, 4]
3
True
```

6. Control Flow (if / loops)

if / elif / else

Executes a block of code only when its condition is True, checking each branch in order.

EXAMPLE

```
age = 20
if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

OUTPUT

```
Adult
```

for loop

Iterates over the items of a sequence (list, string, range, etc.), running the loop body once per item.

EXAMPLE

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)
```

OUTPUT

```
apple
banana
cherry
```

range()

Generates a sequence of numbers, commonly used to control how many times a for loop runs.

EXAMPLE

```
for i in range(1, 6):
    print(i)
```

OUTPUT

```
1
2
3
4
5
```

while loop

Repeats a block of code as long as its condition remains True.

EXAMPLE

```
count = 0
while count < 3:
    print("Count:", count)
    count += 1
```

OUTPUT

```
Count: 0
Count: 1
Count: 2
```

break / continue

break exits a loop immediately; continue skips to the next iteration without exiting.

EXAMPLE

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

OUTPUT

```
0  
1  
2
```

7. Functions & Lambda

def (defining a function)

Groups reusable code under a name. Parameters can have default values, and a function returns a value with the return keyword (or None if omitted).

EXAMPLE

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Ana"))
print(greet("Liam", "Hi"))
```

OUTPUT

```
Hello, Ana!
Hi, Liam!
```

*args and **kwargs

*args collects any number of extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dictionary.

EXAMPLE

```
def total(*args):
    return sum(args)

print(total(1, 2, 3, 4))
```

OUTPUT

```
10
```

lambda

Creates a small, anonymous, single-expression function, often used inline with functions like sorted() or map().

EXAMPLE

```
square = lambda x: x * x
print(square(5))

nums = [3, 1, 4, 1, 5]
print(sorted(nums, key=lambda x: -x))
```

OUTPUT

```
25
[5, 4, 3, 1, 1]
```

8. Comprehensions

Comprehensions build a new list, set, or dictionary from an existing iterable in a single, readable line.

List Comprehension

[expression for item in iterable if condition] — a compact way to transform or filter a list.

EXAMPLE

```
squares = [x**2 for x in range(1, 6)]
evens = [x for x in range(10) if x % 2 == 0]
print(squares)
print(evens)
```

OUTPUT

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
```

Dictionary Comprehension

{key_expr: value_expr for item in iterable} — builds a dictionary in one line.

EXAMPLE

```
nums = [1, 2, 3, 4]
squares = {n: n**2 for n in nums}
print(squares)
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Set Comprehension

{expression for item in iterable} — builds a set, automatically removing duplicates.

EXAMPLE

```
words = ["cat", "dog", "cat", "bird"]
unique_lengths = {len(w) for w in words}
print(unique_lengths)
```

OUTPUT

```
{3, 4}
```

9. Exception Handling

Exception handling lets a program respond to runtime errors gracefully instead of crashing.

try / except

Code in the try block runs first; if an error occurs, control jumps to the matching except block instead of stopping the program.

EXAMPLE

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

OUTPUT

```
Cannot divide by zero!
```

try / except / else / finally

else runs only if no exception occurred; finally always runs, regardless of whether an exception was raised — often used for cleanup.

EXAMPLE

```
try:
    num = int("42")
except ValueError:
    print("Invalid number")
else:
    print("Parsed:", num)
finally:
    print("Done")
```

OUTPUT

```
Parsed: 42
Done
```

raise

Manually triggers an exception, optionally with a custom message, to signal that something invalid has happened.

EXAMPLE

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

set_age(-5)
```

OUTPUT

```
ValueError: Age cannot be negative
```

10. File Handling

open() with a context manager

Opens a file for reading or writing. Using 'with' automatically closes the file when the block ends, even if an error occurs.

FILE MODES

Mode	Description
'r'	Read (default); file must exist
'w'	Write; creates file or overwrites existing content
'a'	Append; adds to the end of the file
'x'	Create; fails if the file already exists

EXAMPLE

```
with open("notes.txt", "w") as f:
    f.write("Hello, file!")

with open("notes.txt", "r") as f:
    content = f.read()
print(content)
```

OUTPUT

```
Hello, file!
```

Reading Line by Line

Iterating directly over a file object yields one line at a time, which is memory efficient for large files.

EXAMPLE

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

11. Object-Oriented Programming

class & __init__

A class is a blueprint for creating objects. `__init__` is the constructor, run automatically when a new object is created, and `self` refers to the instance itself.

EXAMPLE

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def greet(self):
        return f"Hi, I'm {self.name}"

s = Student("Ana", "A")
print(s.greet())
```

OUTPUT

```
Hi, I'm Ana
```

Inheritance

A subclass inherits attributes and methods from a parent class using `class Child(Parent)`, and can override or extend them.

EXAMPLE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, school):
        super().__init__(name)
        self.school = school

s = Student("Ana", "MIT")
print(s.name, s.school)
```

OUTPUT

```
Ana MIT
```

Class Methods & Attributes

Instance methods act on a specific object (`self`); class attributes are shared across every instance of the class.

EXAMPLE

```
class Counter:
    total = 0 # class attribute

    def __init__(self):
        Counter.total += 1

Counter(); Counter(); Counter()
print(Counter.total)
```

OUTPUT

```
3
```

12. Modules & Common Built-ins

import

Loads code from another module (built-in, third-party, or your own file) so its functions and classes can be used.

EXAMPLE

```
import math
from datetime import date
import random as rnd

print(math.sqrt(16))
print(date.today())
print(rnd.randint(1, 10))
```

Useful Built-in Functions

Python includes many general-purpose functions available without any import.

EXAMPLE

```
print(max([3, 7, 2]))
print(min([3, 7, 2]))
print(sum([3, 7, 2]))
print(sorted([3, 7, 2]))
print(list(enumerate(["a", "b"])))
print(list(zip([1, 2], ["a", "b"])))
```

OUTPUT

```
7
2
12
[2, 3, 7]
[(0, 'a'), (1, 'b')]
[(1, 'a'), (2, 'b')]
```

map() / filter()

map() applies a function to every item of an iterable; filter() keeps only the items for which a function returns True. Both return an iterator, often wrapped in list().

EXAMPLE

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
print(doubled)
print(evens)
```

OUTPUT

```
[2, 4, 6, 8]
[2, 4]
```

13. Worked Example Programs

Complete, self-contained programs that combine the concepts above — useful as classroom demonstrations or student exercises.

Program 1: Even or Odd Checker

Reads a number from the user and reports whether it is even or odd using the modulo operator.

EXAMPLE

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print(f"{num} is even.")
else:
    print(f"{num} is odd.")
```

OUTPUT

```
Enter a number: 7
7 is odd.
```

Program 2: Factorial (using a loop)

Calculates $n!$ (the product of all integers from 1 to n) using a for loop and an accumulator variable.

EXAMPLE

```
n = 5
result = 1

for i in range(1, n + 1):
    result *= i

print(f"{n}! = {result}")
```

OUTPUT

```
5! = 120
```

Program 3: Fibonacci Sequence

Generates the first n numbers of the Fibonacci sequence, where each number is the sum of the two preceding ones.

EXAMPLE

```
n = 8
a, b = 0, 1
sequence = []

for _ in range(n):
    sequence.append(a)
    a, b = b, a + b

print(sequence)
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

Program 4: Word Frequency Counter

Counts how often each word appears in a sentence using a dictionary.

EXAMPLE

```
text = "the cat sat on the mat the cat ran"
words = text.split()
frequency = {}

for word in words:
    frequency[word] = frequency.get(word, 0) + 1

print(frequency)
```

OUTPUT

```
{'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}
```

Program 5: Simple Calculator (function-based)

Defines separate functions for each arithmetic operation and dispatches to the correct one based on user input.

EXAMPLE

```
def add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b): return a / b if b != 0 else "Error: divide by zero"

x, y = 10, 4
print("Add:", add(x, y))
print("Subtract:", subtract(x, y))
print("Multiply:", multiply(x, y))
print("Divide:", divide(x, y))
```

OUTPUT

```
Add: 14
Subtract: 6
Multiply: 40
Divide: 2.5
```

Program 6: Class-based Bank Account

Demonstrates OOP by modeling a simple bank account with deposit and withdrawal methods that protect the balance from going negative.

EXAMPLE

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
        else:
            self.balance -= amount

account = BankAccount("Ana", 100)
account.deposit(50)
account.withdraw(30)
```

```
print(f"{account.owner}'s balance: {account.balance}")
```

OUTPUT

```
Ana's balance: 120
```

Program 7: Reading a File and Counting Lines

Opens a text file, counts the number of lines and words, and prints a short summary.

EXAMPLE

```
with open("sample.txt", "r") as f:
    lines = f.readlines()

word_count = sum(len(line.split()) for line in lines)
print(f"Lines: {len(lines)}")
print(f"Words: {word_count}")
```

OUTPUT

```
Lines: 3
Words: 27
```

End of reference guide. This document is intended as a teaching aid — encourage students to type and run each example themselves in a Python interpreter or IDE to see the output firsthand.